

친환경 커머스의 진화

HI.eating

로컬 AI와 3D 시뮬레이션을 접목한
모듈러 홈쇼핑 플랫폼

Spring AI · MCP · RabbitMQ 기반 개인화 마케팅 파이프라인 구축 및 커머스 MVP 구현

2026. 07

유상록 | 이재우 | 공희진 | 전형준
기술 발표 자료

HI.eating 프로젝트

CONTENTS

01.	서론 및 프로젝트 목표	03
02.	시연영상	06
03.	진행방향 및 시스템 설계	07
04.	팀원별 주요 기능 구현	10
05.	협업 및 프로젝트 관리	25
06.	결론	28

서론 및 배경

친환경 식품 커머스와 AI 기반 개인화 마케팅이 필요한 이유

1

소비자 니즈 변화

친환경·유기농 식품 관심 증가
신뢰 가능한 상품 정보와 직관적
탐색 경험 요구

2

기존 커머스 한계

단순 상품 나열 중심 UI

3

AI 도입 비용 부담

SaaS LLM 호출 비용·데이터 외부
전송 리스크
로컬 LLM 기반 실험 필요

문제 제기

커머스 MVP를 넘어 “사용자 행동 데이터 => AI 개인화 콘텐츠 => 비동기 발송”까지 연결되는 실무형 자동화 구조가 필요했습니다.

프로젝트 목표 및 기대 효과

단순 커머스 기능을 넘어 AI·비동기 메시징·3D 체험을 결합한 플랫폼 검증

프로젝트 목표

커머스 MVP 완성

상품 조회·검색·좋아요·구매·리뷰
관리자 핫딜/프로모션/통계 관리

AI 개인화 마케팅

사용자 행동 데이터 기반 추천
Spring AI 핫딜 이메일 초안
자동 생성

실무 기술 검증

RabbitMQ 비동기 발송
MCP·SAM3D·Three.js 등
최신 기술 적용

기대 효과

탐색 비용 절감 · 구매 전환율 향상 · 저비용 로컬 LLM 활용 가능성 검증 · 장애에 강한
비동기 발송 구조 확보

3

Demo

시연영상

사용자 화면과 관리자 화면의 핵심 흐름을 한 번에 확인

The screenshot shows a web browser at localhost:8080/signup. The website has a header with the HI.eating logo, a search bar with the text '건강을 쉽게, HI.eating', and links for '로그인' (Login) and '회원가입' (Sign Up). Below the header is a navigation bar with '카테고리' (Category), '식탁 꾸미기' (Set the table), '찾달' (Find), and user icons. The main content area is titled '회원 가입' (Member Registration) and contains the following form fields:

- 이름** (Name): Input field with placeholder '이름을 입력해주세요.' and a note '* 한글은 10자, 영문은 20자 이내로 입력해 주세요.'
- 이메일** (Email): Input field with placeholder '이메일', a dropdown for '직접입력' (Direct input), and a '중복확인' (Check duplicate) button.
- 비밀번호** (Password): Input field with placeholder '비밀번호를 입력해주세요.' and a note '* 영문, 대소문자, 숫자, 특수문자 중 3개 이상을 조합하여 8자 이상 입력해 주세요.'
- 비밀번호 확인** (Confirm Password): Input field with placeholder '비밀번호를 다시 입력해주세요.'
- 생년월일** (Date of Birth): Input field with placeholder '연도. 월. 일.'
- 성별** (Gender): Radio buttons for '남성' (Male) and '여성' (Female).

At the bottom of the form is a large orange button labeled '회원 가입' (Member Registration). To the right of the form, there is a text prompt: '가입 정보를 차근차근 알려주세요' (Please tell us the registration information step by step).

사용자

상품 탐색 => 즐겨찾기 기반
노출 => 3D 식탁 체험

관리자

재고/유통기한 정렬 =>
찾달 지정 => AI 이메일 생성

운영

통계 대시보드 =>
이메일 발행 상태 확인

진행방향 및 사용 기술

모듈러 모놀리스 구조 위에 AI·비동기·3D 체험 기능을 단계적으로 결합

1

도메인 설계

회원·상품·주문·하달·이메일
로그 중심 ERD 설계

2

커머스 MVP 구현

상품 조회, 좋아요, 구매,
리뷰, 관리자 기능 구현

3

AI/3D 고도화

SAM3D, Three.js, Spring
AI, 로컬 LLM 연동

4

비동기 운영

RabbitMQ, Email Worker,
재시도·누락 검증 설계

사용 도구

Spring Boot

Three.js

Spring AI

MCP

RabbitMQ

GitHub Actions

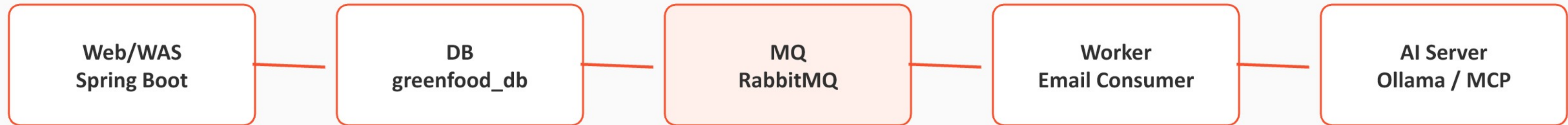
Oracle Text

CodeRabbit

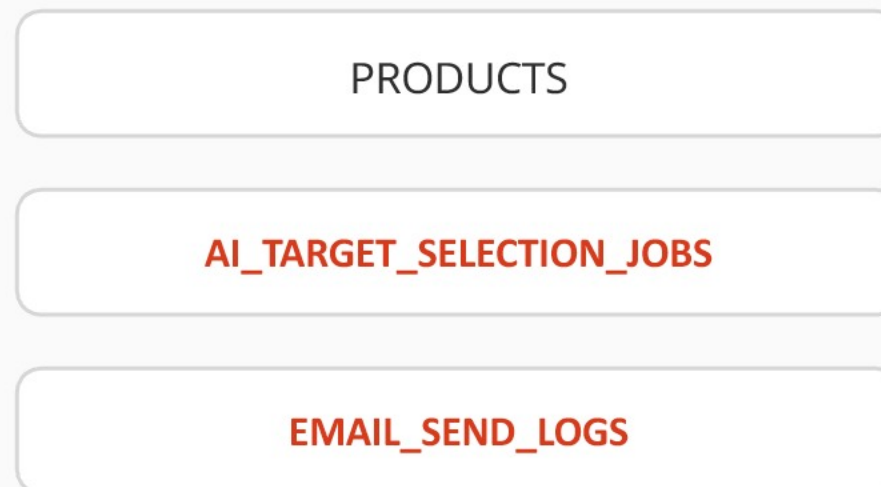
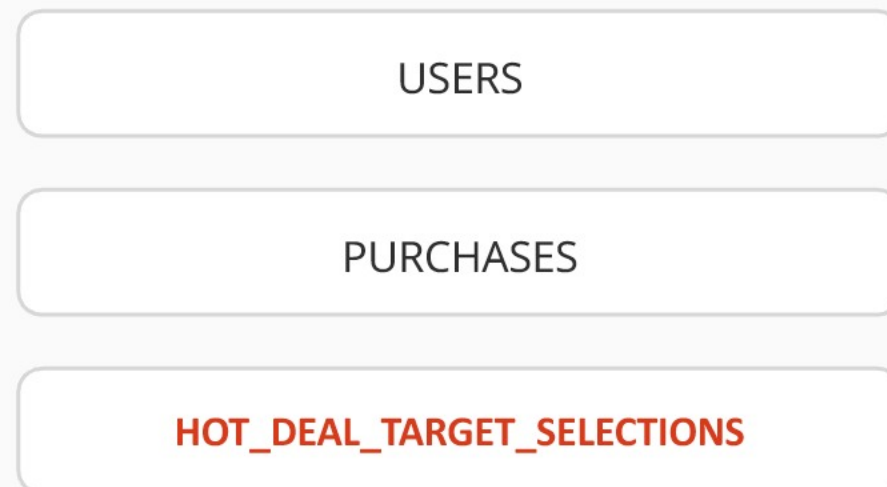
시스템 구조 설계 요약

모듈러 모놀리스 기반으로 DB·MQ·AI 서버를 역할별로 분리

System Architecture



핵심 데이터 모델



핵심 시퀀스

로그인/권한 분기
→ 상품 탐색/좋아요
→ 재고 차감 구매
→ 관리자 핫딜 지정
→ AI 이메일 발송

3D 시뮬레이션 - SAM 3D

2D 음식 사진을 3D 자산으로 전환하고, 사용자 행동 데이터 기반 추천 경험을 구현

1

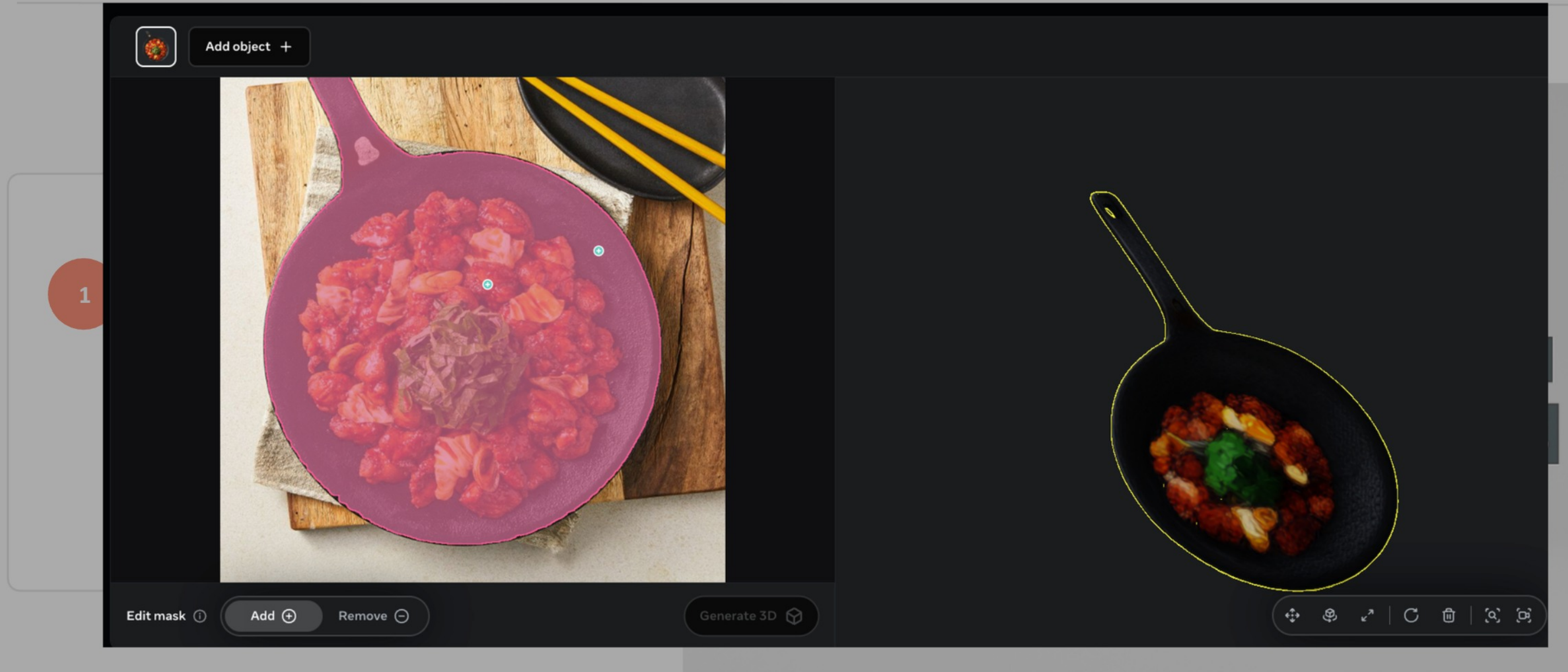
SAM3D AI API

업로드된 2D 음식 사진을
3D 애셋으로 자동 변환



3D 시뮬레이션 - SAM 3D

2D 음식 사진을 3D 자산으로 전환하고, 사용자 행동 데이터 기반 추천 경험을 구현



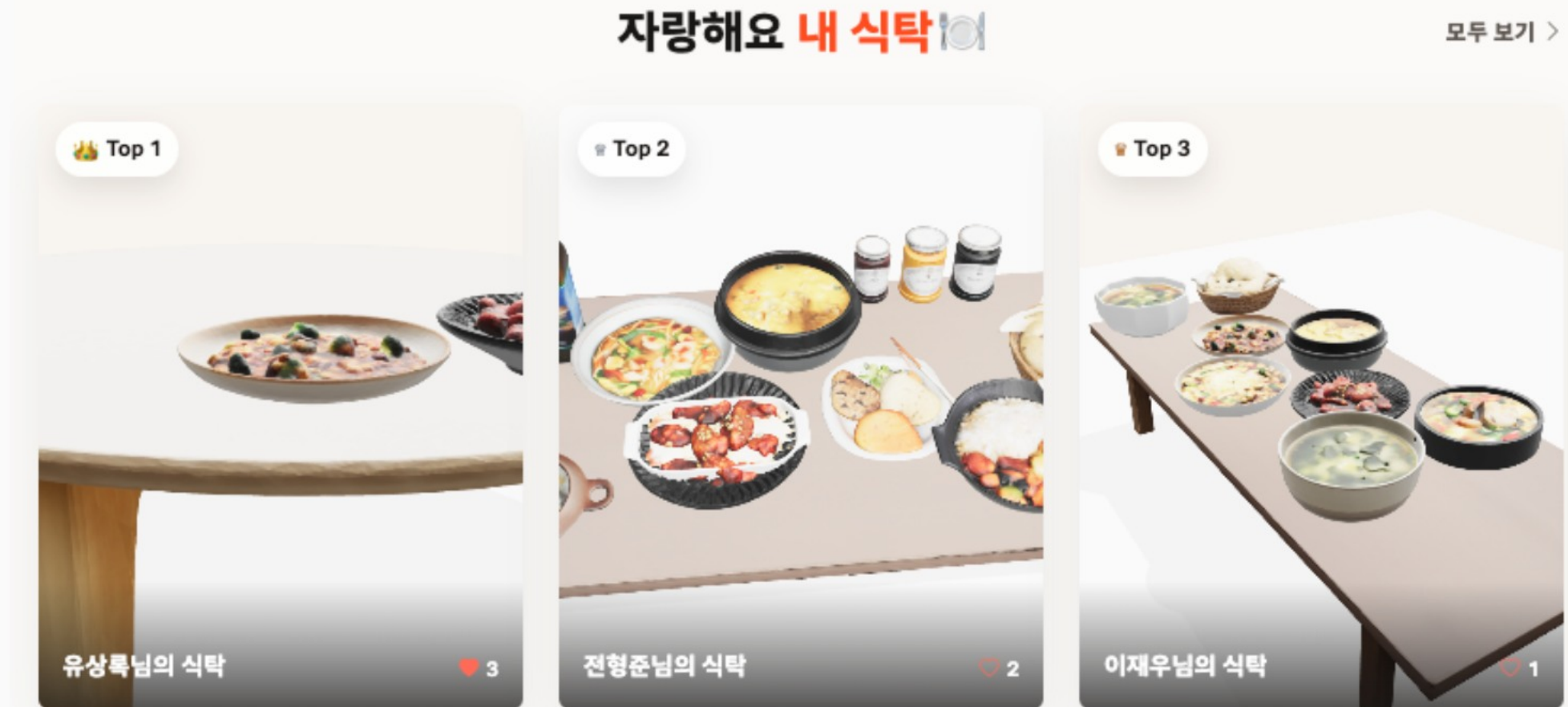
3D 시뮬레이션 - Three.js

2D 음식 사진을 3D 자산으로 전환하고, 사용자 행동 데이터 기반 추천 경험을 구현

2

Three.js 가상 식탁

변환된 3D 식품 데이터를
브라우저에서 배치·체험하는
식탁 시뮬레이션 구현



사용자 경험: “상품을 보기만 하는 커머스”에서
“식탁 위에 직접 배치해보는 커머스”로 확장

로컬 AI 기반 상품 추천

2D 음식 사진을 3D 자산으로 전환하고, 사용자 행동 데이터 기반 추천 경험을 구현

3 로컬 AI + 벡터 DB

사용자 행동 로그 임베딩을 통한
개인화 상품 추천 알고리즘 설계

1

Spring AI

OllamaEmbeddingModel · OllamaChatModel · OllamaApi
spring-ai-bom 1.1.8 — LLM 연결을 단일 인터페이스로 추상화

2

Local LLM (Ollama)

로컬 구동 · 3개 독립 인스턴스
:11434 생성 · :11435 검증 · :11436 임베딩 (nomic-embed-text)

nomic-embed-text 구동

3

Embedding Model (nomic-embed-text)

텍스트 → 고차원 수치 벡터 변환
"한우불고기" → [0.023, -0.452, 0.871, ...]

4

Cosine Similarity

두 벡터 간 각도 기반 유사도 측정
사용자 프로필 × 각 상품 벡터 간 유사도 점수로 순위 산출

Data Flow

Spring AI → Ollama (추상화) → Embedding Model (벡터화) → Cosine Similarity (유사도 측정) → 추천 결과

로컬 AI 기반 상품 추천

2D 음식 사진을 3D 자산으로 전환하고, 사용자 행동 데이터 기반 추천 경험을 구현

STEP 1 · 사용자 프로필 벡터 생성

구매
purchases (w=3.0)

찜
favorites (w=2.0)

방문
visits (w=1.0)

고평점 리뷰
reviews (w=1.5)

각 상품명을 동일한 nomic-embed-text로 벡터화
가중평균 정규화 → 하나의 사용자 프로필 벡터

STEP 2 · 코사인 유사도 계산 및 TOP N 선정

사용자 프로필 벡터
UserVector (768차원)

×

각 상품의 임베딩 벡터
ProductVector₁ · ProductVector₂ · ...

→

Cosine Similarity
유사도 점수 리스트

→

TOP 7 선정
유사도 순 정렬 → 상위 7개

$$\text{similarity} = \cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|) \text{ — 값이 1에 가까울수록 유사함}$$

STEP 3 · 추천 결과 반환

유사도가 가장 높은 7개의 상품을 사용자에게 보여준다. 임베딩 기반이므로 사용자가 구매/찜한 상품과
「비슷한 이름/카테고리」의 상품이 추천된다 (협업 필터링 대신 콘텐츠 기반 유사도 방식)

크롤러 : 시스템 데이터 품질 향상의 목적

현대그린푸드의 greating 크롤링 진행

1

카테고리 파싱

Jsoup으로 depth1/depth2
링크를 읽고 ctgryId를 추출

2

수집 범위 제한

설정 파일의 6개 대표 카테고리
기준으로 대상 필터링

3

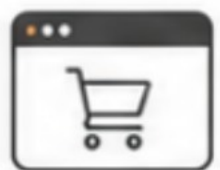
상품 API 호출

/biz/market/getGoodsList에
page, sPage/ePage, AJAX 헤더
전달

4

DTO 매핑

이름·가격·이미지·용량·칼로리·
보관 정보를 Product 모델로
변환



Greating
카테고리/상품



HTML 파싱
+ 상품 API 호출



Oracle DB 저장
카테고리, 상품
이미지, 옵션



수동 seed 없이 실제 상품 데이터 확보

사용 도구

Java17

JSoup

MAVEN

Spring Security : 로그인 이후 모든 기능의 권한 기준

사용자 화면, 관리자 화면, API 호출, 정적 리소스 접근을 하나의 필터 체인에서 통제

1

BCrypt PasswordEncoder

회원가입 시 BCrypt 해시 저장, 로그인 시 해시 기반 검증

2

Custom UserDetails

HiEatingUserPrincipal에 userId, email, name, authorities를 담아 전역에서 사용

3

Role Routing

/admin/**는 ROLE_ADMIN만 접근, 일반 사용자는 사용자 화면으로 분기

4

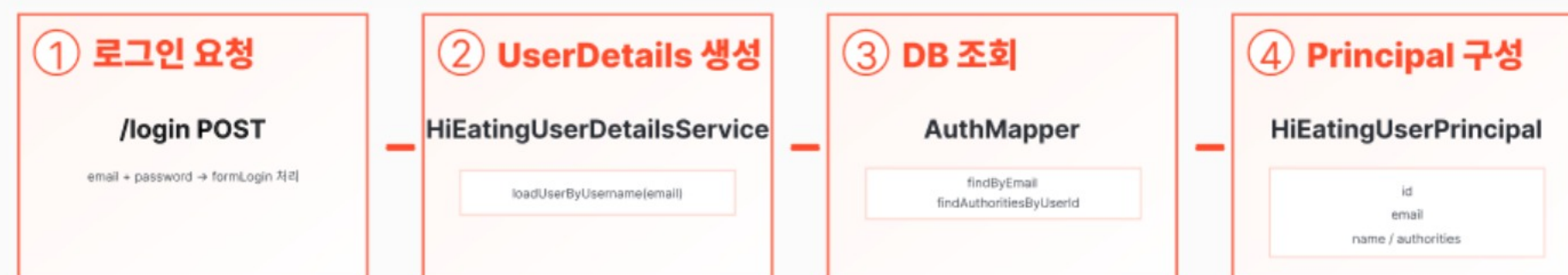
CSRF

Thymeleaf form과 fetch API 요청 모두 CSRF 토큰을 포함

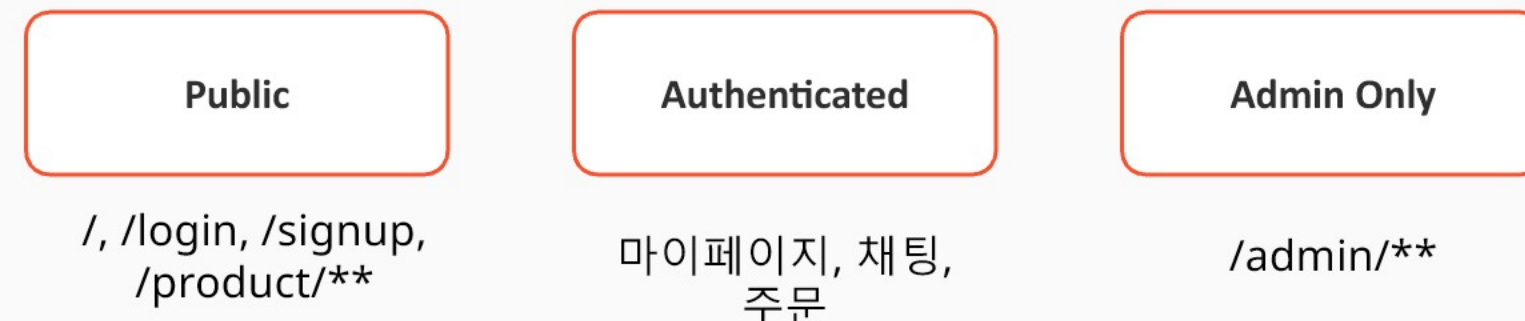
인증 흐름 : 세션, Principal, CSRF가 함께 동작

로그인 성공 후 DB 사용자 id를 꺼낼 수 있는 Principal을 만들어 다른 기능에서 재사용

① 세션 기반 로그인 / Principal 구성



② 요청 경로별 권한 분리



1. Principal에 DB userId와 권한을 담아 **WebSocket, 마이페이지, 관리자 기능**에서 같은 인증 정보를 재사용
2. CSRF는 CookieCsrfTokenRepository로 유지하고, JS fetch는 X-XSRF-TOKEN 헤더로 전달

성과: USER/ADMIN 권한 분기와 CSRF 보호를 유지하면서 서비스 전역 인증 기반을 통일

MCP Server : AI 클라이언트의 H.eating을 도구화

Ollama 연동이 아니라, 외부 MCP 클라이언트가 호출할 서버 도구를 만드는 것

1

직접 DB 노출 방지

ChatGPT가 SQL을 실행하는 대신
서비스 계층 메서드를 도구로 호출

2

도구 계약 명확화

@McpTool 이름·설명·파라미터로
호출 가능한 기능을 명시

3

분석 결과 구조화

리뷰 요약, 평점 기반 감정 분류,
대표 리뷰 발제를 응답 모델로
반환

상품 탐색

list_categories
search_products
list_products_by_category
get_product_detail

홈·핫딜

list_most_purchased_products
list_active_hot_deals
list_hot_deal_products

프로모션

list_active_promotions
list_active_hot_deal_products
핫딜 상품 목록 조회

리뷰 분석

list_product_reviews
get_product_review_summary
analyze_product_reviews

AI 모델을 붙이는 것보다 'AI가 안전하게 읽을 수 있는 서비스 도구 표면'을 설계하는 것이 중요했습니다.

MCP : Annotation 기반 도구 등록 및 서비스 위임

Spring AI MCP annotation scanner가 @McpTool을 읽고 서버 도구로 노출

① @McpTool 기반 서버 도구 등록

HiEatingMcpTools.java

비즈니스 로직 재작성 없이
기존 Service 계층으로 위임

② ChatGPT/MCP Client 호출 흐름

Client 요청

Tool 선택

Service 호출

DTO 응답

리뷰 요약

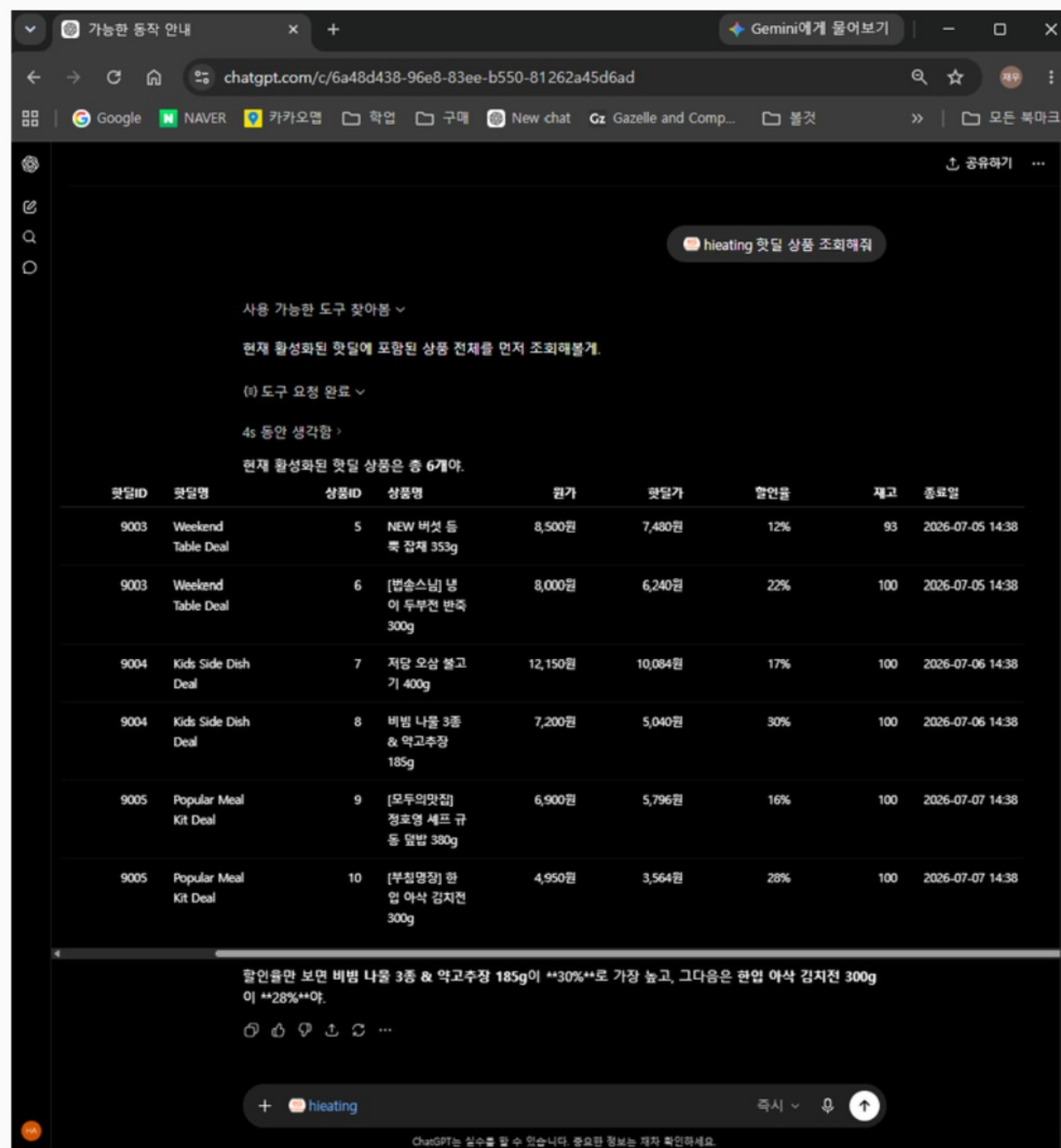
분석 반환

핵심 설계: MCP 서버는 직접 DB 접근을 열지 않고, 기존 Category/Product/HotDeal/Review Service를 재사용

성과: ChatGPT가 상품 검색과 리뷰 분석을 서비스 API 수준에서 호출할 수 있는 기반 확보

MCP 개발 효과

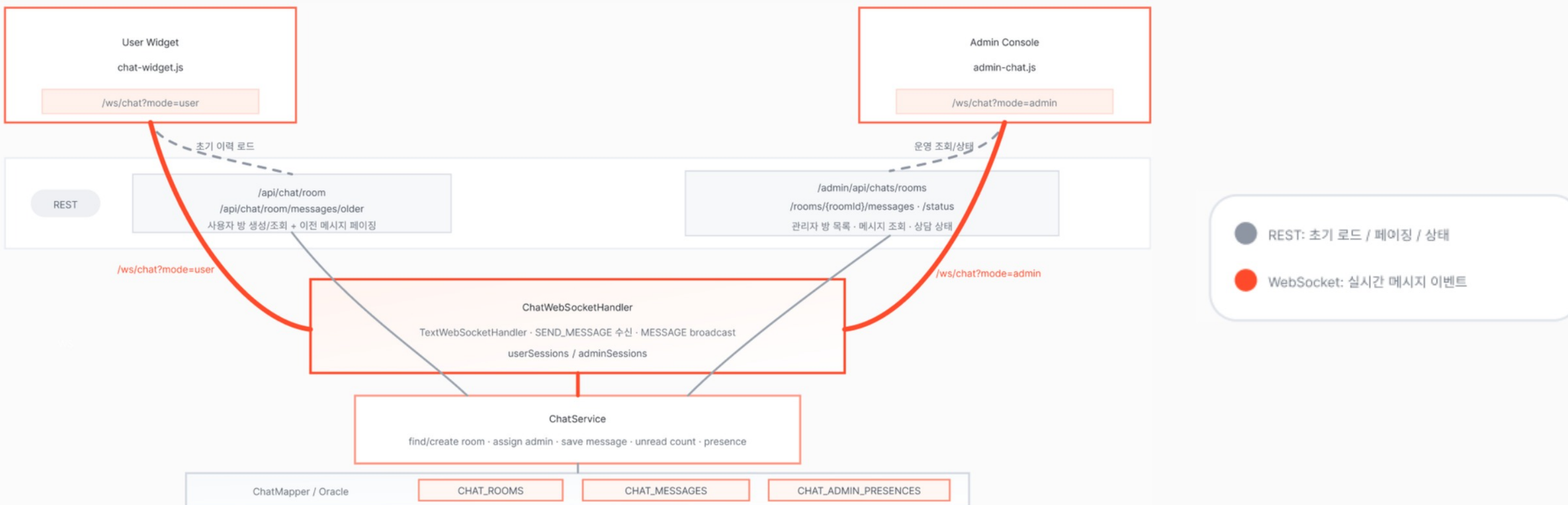
시스템 접근성 향상 및 사용 시간 효율화



REST : 채팅 이력 / WebSocket : 실시간 이벤트를 처리

초기 로딩과 실시간 메시지를 분리해 안정성과 사용자 경험을 함께 확보

Chat Architecture



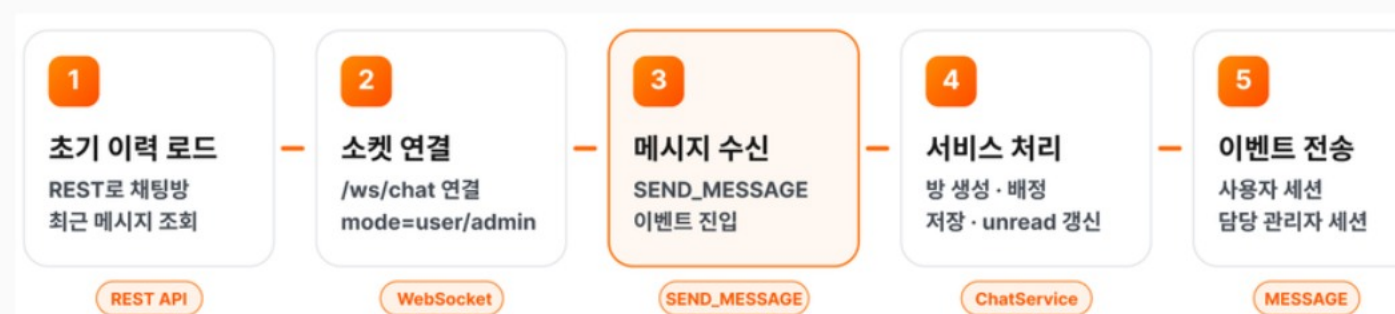
핵심 테이블

CHAT_ROOMS

CHAT_MESSAGES

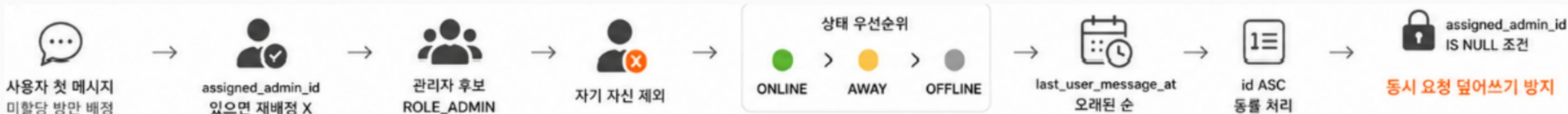
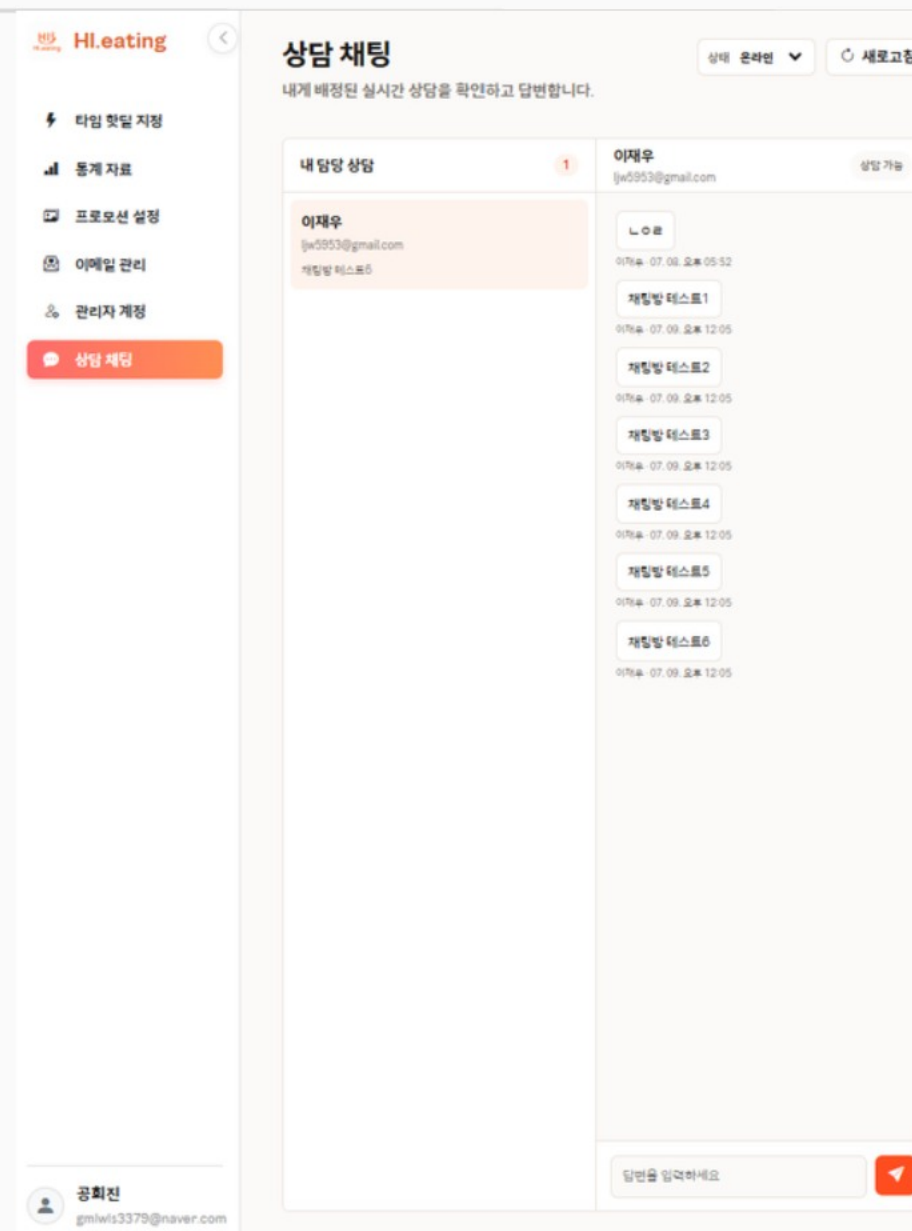
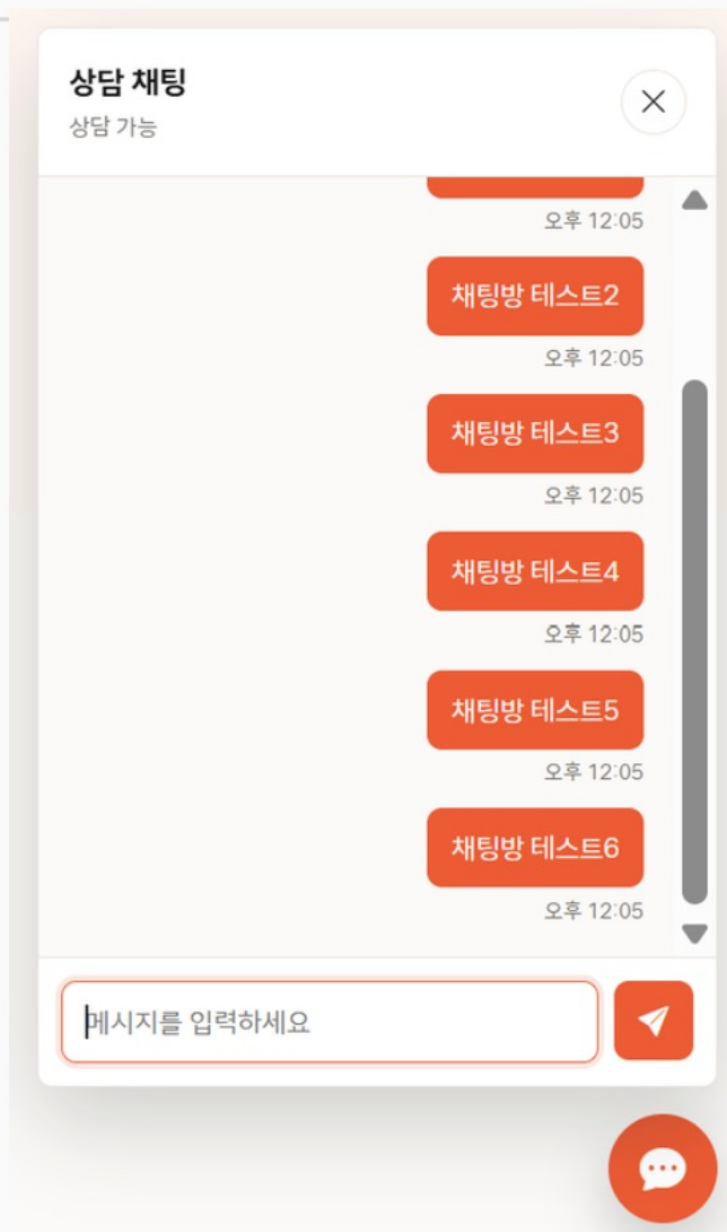
CHAT_ADMIN_PRESENCES

핵심 시퀀스



사용자 채팅 관리자 배정 알고리즘 설계

일반적인 개인-개인 채팅이 아닌, 사용자-관리자 채팅의 특성을 고려한 관리자 배정 알고리즘 설계



부하 분석 및 개선 - K6

전체 500개 메시지 초기 로딩 -> 최근 50개 + cursor paging으로 변경

1

초기 채팅방 조회 결과

지표	전체 로딩	50개 로딩	변화
평균 응답	820ms	279ms	약 66% 개선
p95 응답	2073ms	647ms	약 69% 개선
응답 크기 평균	87.8KB	9.3KB	약 89% 감소

2

WebSocket 실시간 테스트

지표	전체 로딩	50개 로딩	변화
echo latency avg	1809ms	472ms	약 74% 개선
echo latency p95	6423ms	1196ms	약 81% 개선
초기 payload	87.4KB	9.3KB	약 89% 감소



```

execution: local
script: load-tests\chat-realtime-flow.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 5m30s max duration (incl. graceful stop):
  * user_chat: 10 looping VUs for 5m0s (exec: userChat, gracefulStop: 30s)

THRESHOLDS

chat_user_echo_latency_ms
  x 'p(95)<1000' p(95)=1195.55

chat_user_ws_connected
  ✓ 'rate>0.95' rate=100.00%

chat_ws_errors
  ✓ 'count<1' count=0

checks
  ✓ 'rate>0.95' rate=100.00%

http_req_failed
  ✓ 'rate<0.05' rate=0.00%

TOTAL RESULTS

checks_total.....: 70      0.232232/s
checks_succeeded...: 100.00% 70 out of 70
checks_failed.....: 0.00%   0 out of 70

✓ user login succeeded
✓ user chat room loaded
✓ user websocket upgraded

CUSTOM
chat_messages_received.....: 2972    9.859917/s
chat_messages_sent.....: 2979    9.883141/s
chat_user_echo_latency_ms.....: avg=472.243729 min=80      med=349    p(90)=973.2    p(95)=1195.55    p(99)=1761.18    max=2576
chat_user_room_has_more.....: 100.00% 10 out of 10
chat_user_room_messages_count.....: avg=50      min=50      med=50      p(90)=50      p(95)=50      p(99)=50      max=50
chat_user_room_payload_bytes.....: avg=9263    min=9253    med=9254    p(90)=9263.3    p(95)=9385.15    p(99)=9338.63    max=9347
chat_user_ws_connected.....: 100.00% 50 out of 50
chat_ws_errors.....: 0      0/s

HTTP
http_req_duration.....: avg=624.24ms min=304.57ms med=602.4ms p(90)=939.89ms p(95)=948.54ms p(99)=961.02ms max=964.14ms
{ expected_response:true }.....: avg=624.24ms min=304.57ms med=602.4ms p(90)=939.89ms p(95)=948.54ms p(99)=961.02ms max=964.14ms
http_req_failed.....: 0.00%   0 out of 20
http_reqs.....: 20      0.066352/s

EXECUTION
iteration_duration.....: avg=1m0s    min=1m0s    med=1m0s    p(90)=1m1s    p(95)=1m1s    p(99)=1m1s    max=1m1s
iterations.....: 50      0.16588/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 2.1 MB    7.0 kB/s
data_sent.....: 313 kB    1.0 kB/s

WEBSOCKET
ws_connecting.....: avg=28.77ms min=3.07ms med=7.63ms p(90)=102.64ms p(95)=138.34ms p(99)=151.62ms max=164.38ms
ws_msgs_received.....: 2972    9.859917/s
ws_msgs_sent.....: 2979    9.883141/s
ws_session_duration.....: avg=1m0s    min=1m0s    med=1m0s    p(90)=1m0s    p(95)=1m0s    p(99)=1m0s    max=1m0s
ws_sessions.....: 50      0.16588/s
  
```


시스템 톤 조절 및 UX 개선

사용자가 폼을 끝까지 진행하도록 돕는 장치 / 시스템 전체 분위기 구성

1 Lottie란?

JSON 기반 **벡터 애니메이션**을 웹에서 SVG/Canvas로 렌더링하는 방식

2 적용 근거 및 효과

입력 단계가 많은 인증 화면에서
사용자가 현재 해야 할 행동을 부드럽게 안내

GIF/영상보다 선명하고 상태 전환이
쉬우며 브랜드 캐릭터의 감정 표현을
살림

회원 가입

이름

이름을 입력해주세요

* 한글은 10자, 영문은 20자 이내로 입력해 주세요

이메일

이메일

@

직접입력

▼

중복확인

비밀번호

비밀번호를 입력해주세요

* 영문, 대소문자, 숫자, 특수문자 중 3개 이상을 조합하여 8자 이상 입력해 주세요

비밀번호 확인

비밀번호를 다시 입력해주세요

생년월일

연도-월-일

□

성별

☐

남성

☐

여성

회원 가입

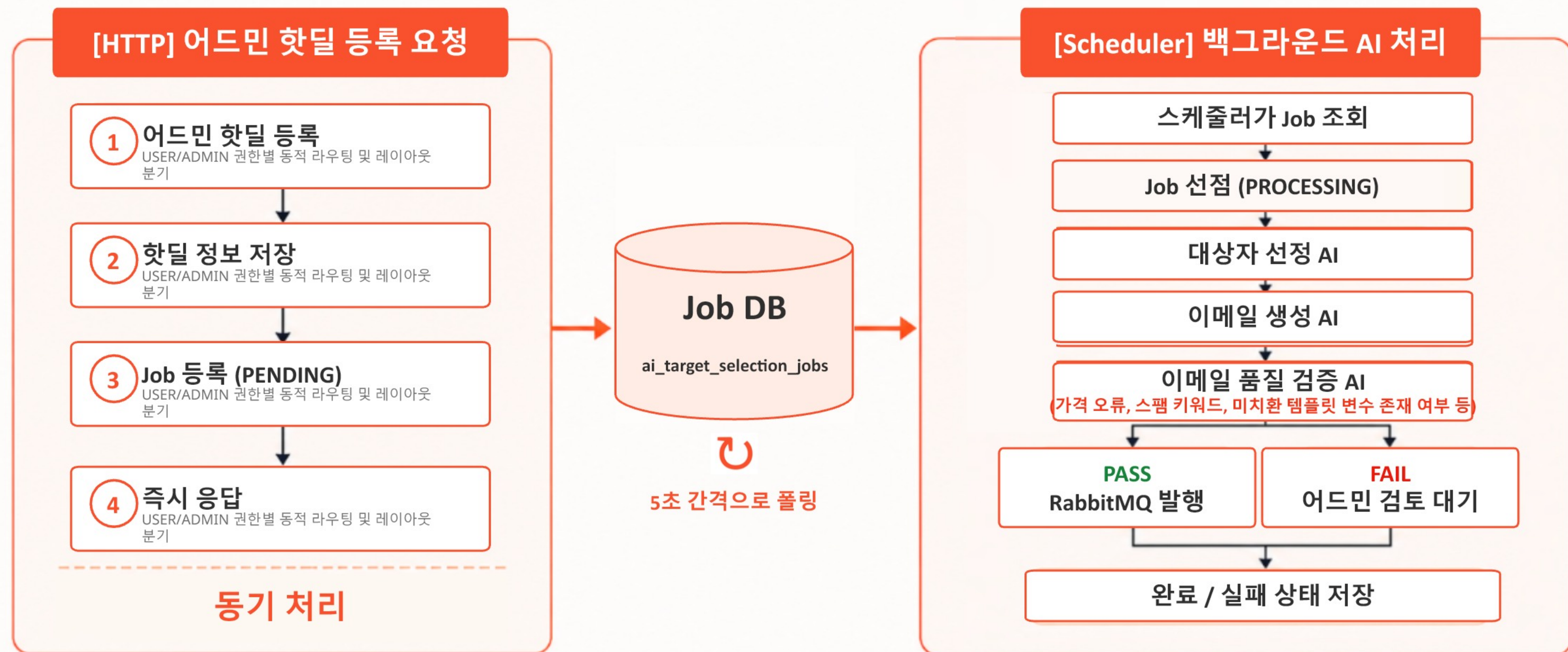


반갑습니다!
이름을 알려주세요 😊

성과: 로그인/회원가입 압력 상황에 맞는 반응형 마스코트로 인증 화면의 친근함 안내성 강화

Spring AI 핫딜 이메일 자동화 파이프라인

관리자 운영 효율화와 AI 기반 핫딜 이메일 생성 자동화 파이프라인



HTTP는 빠른 응답을 위해 동기 처리, AI는 지연·실패에 대비해 비동기 처리
 “응답 속도 향상 · 장애 격리 · 재시도 가능한 구조 확보”

오픈소스 Ollama 기반 LLM 도입 배경

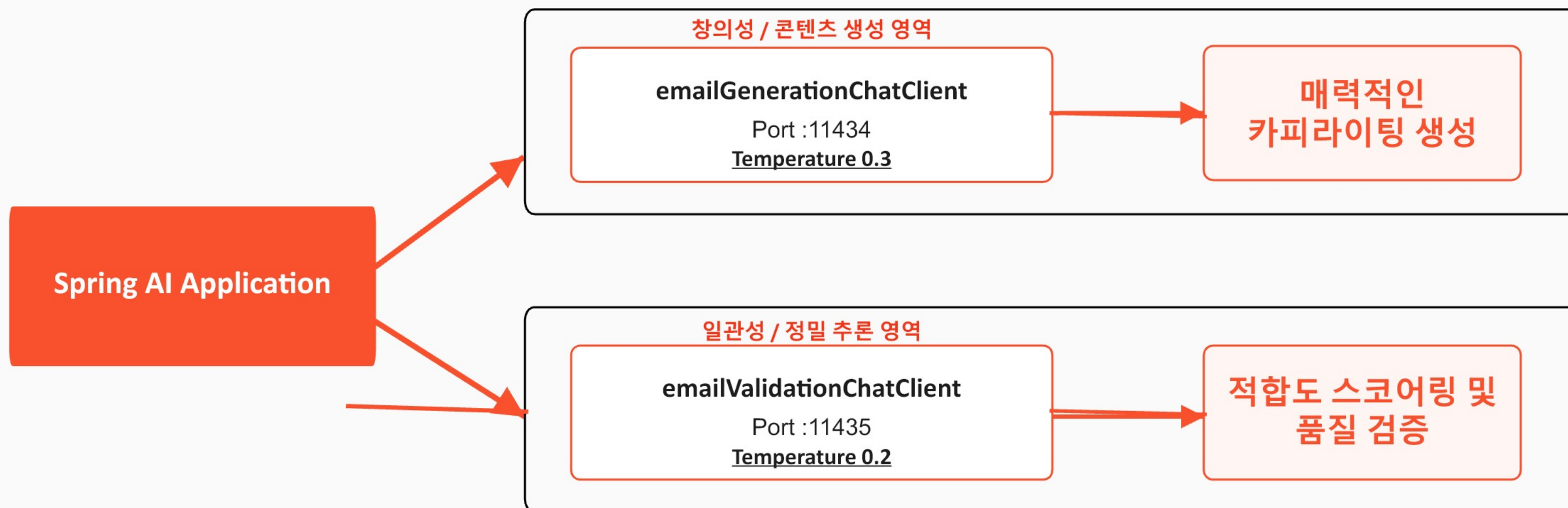
실제 인프라 명세와 모델 비교를 통해 자체 원격 Ollama 구조 선택

모델명 (규모)	강점 / 추천 용도	특징 및 선정 Rationale	프로젝트 적합도
Llama 3.2 3B (3.8B)	올라운더, 이메일 마케팅 문구 생성	한국어 성능 밸런스 우수, Spring AI 연동 예제 풍부	★★★★★ 최종 채택
Phi-4 Mini (3.8B)	논리적 추론력, 구매 이력 기반 추천	파라미터 대비 추론 능력 우수, 7B급 수준 추론 성능	★★★★☆
Gemma 3 2B (2B)	압도적 속도, 실시간 리뷰 요약	가볍고 빠르며 CPU 환경에서도 높은 토큰 생성 속도	★★★★☆
Qwen 3 7B (7B)	다국어 및 문맥, 고품질 마케팅 콘텐츠	한국어 포함 다국어 강점, 다만 높은 RAM 점유 고려	★★★★☆

한국어 품질, 운영 편의성, 비용 예측 가능성을 고려해 Ollama 표준 모델로 채택

이중 ChatClient 설정을 통한 성능 및 장애 격리

이메일 생성용과 검증용으로 AI 서버 역할 분리

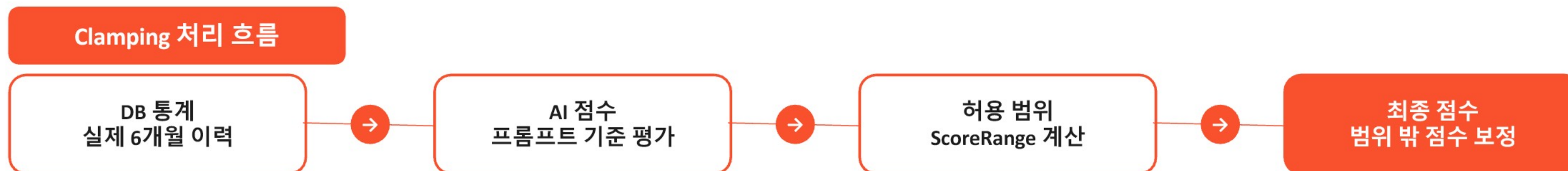


이메일 문구는 더 자연스럽게 생성하고, 적합도 스코어링과 품질 검증은 일관된 기준으로 안정적으로 수행

AI 환각 방어 : 핫딜 이메일 발송 대상 선정 Score Clamping

AI가 제안한 점수는 참고값으로 사용하고, Java/DB 통계로 최종 점수 범위를 강제 보정

Clamping 처리 흐름



예시 A 과대평가

찜 1회 → 실제 범위 0~59점
AI 95점 → Clamp 결과 59점 → 탈락

예시 B 과소평가

구매 4회+리뷰 → 실제 범위 90~100점
AI 75점 → Clamp 결과 90점 → 선정

Score Clamping으로 AI는 선호도 판단을 돕고, Java/DB가 실제 데이터 기반으로 최종 점수를 통제

AI 환각 방어 : 핫딜 이메일 대상 선정 작동 기준

AI는 선호도 판단을 돕고, 최종 발송 여부는 Java/DB 기준으로 통제 → 안정성 확보

Clamping 처리 흐름

DB 통계
실제 6개월 이력

AI 점수
프롬프트 기준 평가

허용 범위
ScoreRange 계산

최종 점수
범위 밖 점수 보정

① DB 지표 수집

최근 6개월 실제 행동 데이터

• purchaseCount : 구매 횟수

• reviewCount : 리뷰 횟수

• favoriteCount : 즐겨찾기 횟수

• 활동 카테고리 매칭

구매 = 강한 신호 / 리뷰·찜 = 보조 신호

② AI 채점 가이드라인

Ollama 프롬프트 기준

점수	AI 판단 기준
90~100	구매 3회+ + 리뷰/찜
80~89	구매 2회+ 또는 구매+리뷰+찜
60~79	구매 1회+
0~59	단순 찜·리뷰 또는 실적 없음
AI는 비즈니스 규칙에 따른 1차 점수만 제안	

③ Java Clamping 검증

TargetScorePolicy 강제
보정

실제 DB 이력 기준으로 허용 범위 계산

ScoreRange(min, max)



finalScore = max(min, min(max, aiScore))

핫딜 이메일 발송 대상 선정

finalScore >= scoreThreshold(80점)

AI 환각 방어 : 핫딜 이메일 대상 선정 작동 기준

AI는 선호도 판단을 돕고, 최종 발송 여부는 Java/DB 기준으로 통제 → 안정성 확보

Clamping 처리 흐름

DB 통계
실제 6개월 이력

AI 점수
프롬프트 기준 평가

허용 범위
ScoreRange 계산

최종 점수
범위 밖 점수 보정

```
--- [hi-eating] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 24 ms
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetSelectionJobProcessor : [대상선정 Job] 시작 jobId=83 hotDealId=102 retryCount=0/3
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetUserSelectionAiService : [대상선정] 후보 조회 완료 hotDealId=102 후보=25명 최대=30명
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetUserSelectionAiService : [대상선정] AI 배치 평가 시작 hotDealId=102 batch=1/1 users=25
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetUserSelectionAiService : [대상선정] AI 배치 평가 완료 hotDealId=102 batch=1/1 users=25 elapsedMs=70439
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetUserSelectionAiService : [대상선정] 완료 hotDealId=102 후보=25명 선정=14명 저장=14명
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetSelectionJobProcessor : [대상선정 Job] 이메일 생성 시작 jobId=83 hotDealId=102
--- [hi-eating] [ scheduling-2] k.o.h.a.s.HotDealEmailGenerationService : [이메일 생성] 완료 hotDealId=102 발송로그=14건
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetSelectionJobProcessor : [대상선정 Job] 이메일 검증 시작 jobId=83 hotDealId=102
--- [hi-eating] [ scheduling-2] a.s.HotDealEmailQualityValidationService : [HotDeal Email Validation] completed hotDealId=102 result=PASS applied=14
--- [hi-eating] [ scheduling-2] k.o.h.a.s.TargetSelectionJobProcessor : [대상선정 Job] 완료 jobId=83 hotDealId=102 후보=25 선정=14 저장=14 소요시간(ms): 대상선정=70605, 이메일생성=18559, 이메일검증=39922, 전체=129095
--- [hi-eating] [ scheduling-4] k.o.h.email.publisher.EmailPublisher : [이메일 처리시간] RabbitMQ 발행 직전 emailDraftId=452 hotDealId=102 핫딜생성부터발행까지=127830ms(127.830초)
--- [hi-eating] [ scheduling-4] o.s.a.r.c.CachingConnectionFactory : Attempting to connect to: [100.87.22.127:5672]
--- [hi-eating] [ scheduling-4] o.s.a.r.c.CachingConnectionFactory : Created new connection: rabbitConnectionFactory#3bab54c5:0/SimpleConnection@437d1561 [delegate=amqp://hdf@100.87.22.127:5672/, localPort=52467]
--- [hi-eating] [ scheduling-4] k.o.h.email.publisher.EmailPublisher : [이메일 처리시간] RabbitMQ 발행 확인 완료 emailDraftId=452 hotDealId=102
--- [hi-eating] [ scheduling-4] k.o.h.email.publisher.EmailPublisher : [이메일 처리시간] RabbitMQ 발행 직전 emailDraftId=453 hotDealId=102 핫딜생성부터발행까지=128088ms(128.088초)
```

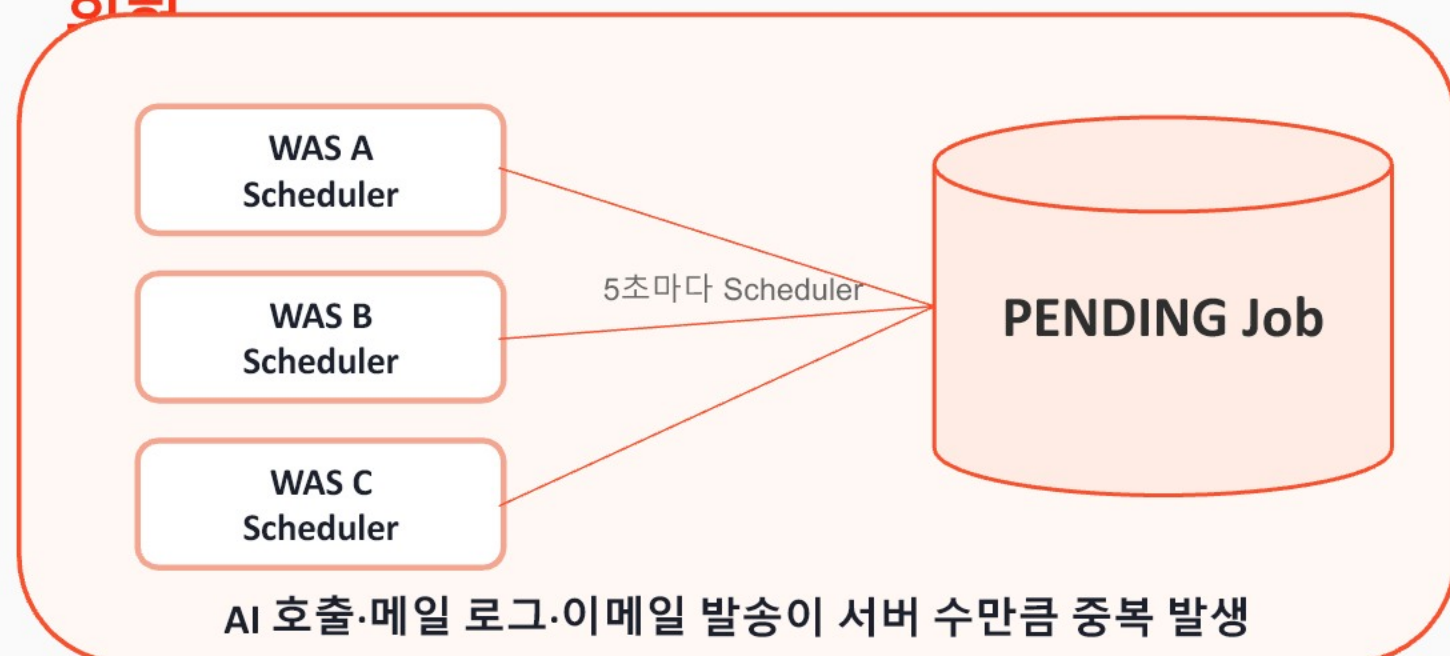
핫딜 이메일 발송 대상 선정

$\text{finalScore} \geq \text{scoreThreshold}(80\text{점})$

분산 환경 동시성 제어: 낙관적 락

분산 락 없이도 DB 원자성으로 중복 처리/중복 발송을 원천 차단

① 분산 환경 문제 상황: 스케일 아웃 시 중복 처리



② 기존 방식의 한계와 우리의 선택

Redis 분산 락

Redis 인프라 추가
운영 복잡도 증가

비관적 락 SELECT ... FOR UPDATE

AI 호출 30초 동안
DB 커넥션 Blocking

낙관적 락(SQL CAS)

추가 인프라 없음
실패 시 즉시 return

핫딜 Job은 충돌 빈도가 낮고, 이후 AI 호출 시간이 길기 때문에 락을 오래 잡는 방식보다 “선점 실패 시 즉시 포기”가 최적

③ SELECT 후 UPDATE 대신, UPDATE 1번으로 Compare + Swap

일반 방식

SELECT

UPDATE

DB 2회 통신 + Race Condition 틈새

```
UPDATE ai_target_selection_jobs
SET status = 'PROCESSING'
WHERE id = #{jobId} AND status = 'PENDING'
```

WHERE 절이 Compare, SET 절이 Swap 역할

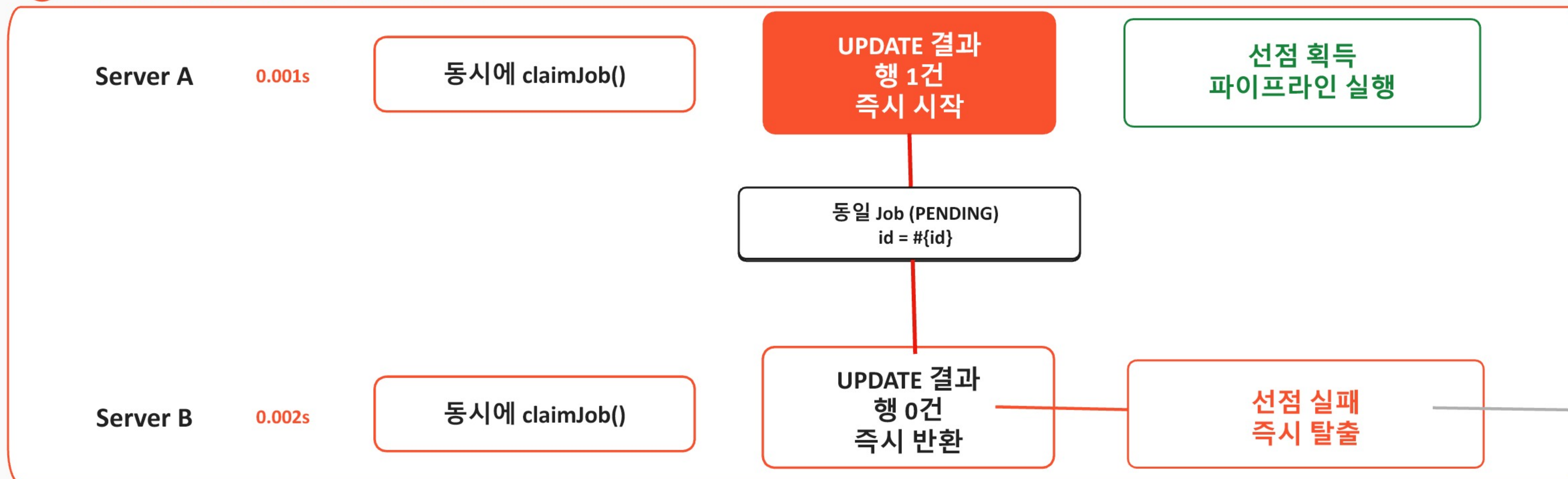
별도의 분산 락 인프라 없이, 데이터베이스 업데이트 쿼리 한 줄의 원자성을 이용

분산 환경 동시성 제어: 낙관적 락 선점

분산 락 없이도 DB 원자성으로 동일 Job의 중복 처리와 중복 발송을 차단

```
UPDATE ai_target_selection_job  
SET status = 'PROCESSING' WHERE id = #{id} AND status = 'PENDING'
```

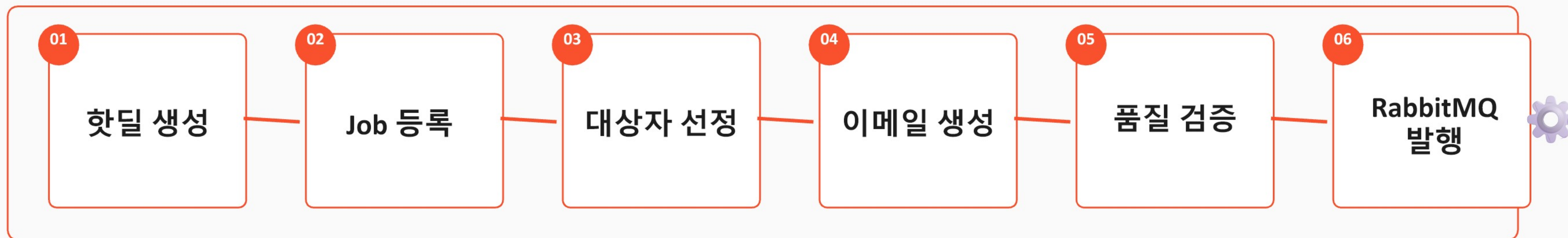
④ 결과는 “영향받은 행 수”로 판단



별도 분산 락 없이도 중복 발송을 막고, 실패한 서버 스레드는 대기 없이 빠르게 종료

Spring AI 핫딜 이메일 파이프라인

핫딜 등록부터 검증 및 발행까지의 무중단 자동화



LIKE → Oracle Text 검색 성능 개선

로컬 더미 1,000,000건 비교 테스트를 바탕으로 검색어 유형별 전략을 조정

테스트 기준

1,000,000건

로컬 더미 상품 데이터

기존 검색

LIKE '%keyword%'



개선 검색

CONTAINS(name, ...)

LIKE → Oracle Text 검색 성능 개선

로컬 더미 1,000,000건 비교 테스트를 바탕으로 검색어 유형별 전략을 조정

검색 정책

01 한 글자 검색

Oracle Text가 LIKE보다 불리
프론트에서 2글자 이상 검색 제한

02 부분 문자열

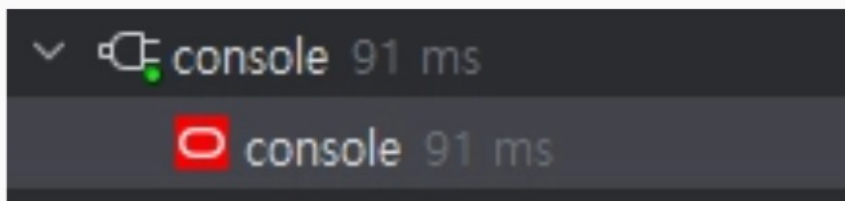
일반 Oracle Text는 토큰 단위 한계
SUBSTRING_INDEX / PREFIX_INDEX 적용

03 실제 상품 검색어

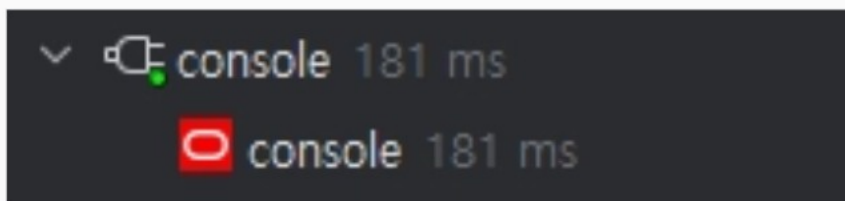
토큰·식별 가능한 문자열에서 효과 기대
상품명 검색은 Oracle Text 중심으로 전환



LIKE



Oracle Text



인덱스 재구성

**PRODUCT_WORDLIST + SUBSTRING_INDEX +
PREFIX_INDEX**

예: 돼지불고기 안의 “불고기”, 닭가슴살 안의 “닭” 검색 대응

한 토큰 검색

search?keyword=%EB%BB%AD%EA%B0%80%EC%8A%B4%EC...	200	document	Other	1,033 kB	2.42 s	2.42s
search?keyword=%EB%BB%AD%EA%B0%80%EC%8A%B4%EC...	200	document	Other	1,033 kB	1.41 s	1.41s

식별 가능 문자 검색

search?keyword=00100	200	document	Other	11.6 kB	1.95 s	1.95s
search?keyword=00100	200	document	Other	11.6 kB	162 ms	162ms

LIKE → Oracle Text 검색 성능 개선

로컬 더미 1,000,000건 비교 테스트를 바탕으로 검색어 유형별 전략을 조정

검색 정책

01 한 글자 검색

Oracle Text
프론트에서 2

02 부분 문자열

03 실제 상품 검색어

와 기대
으로 전환

단순 Oracle Text로의 전환이 아닌,
테스트 결과에 따라 **검색어 길이**와 **유형**별로 프로젝트에 맞게 전략
구성

LIKE

console 91 ms
console 91 ms

Oracle Text

console 181 ms
console 181 ms

인덱스 재구성

PRODUCT_WORDLIST + SUBSTRING_INDEX +
PREFIX_INDEX

예: 돼지불고기 안의 “불고기”, 닭가슴살 안의 “닭” 검색 대응

search?keyword=%EB%BB%AD%EA%B0%80%EC%8A%B4%EC...	200	document	Other	1,033 kB	2.42 s	2.42s
search?keyword=%EB%BB%AD%EA%B0%80%EC%8A%B4%EC...	200	document	Other	1,033 kB	1.41 s	1.41s

식별 가능 문자 검색

search?keyword=00100	200	document	Other	11.6 kB	1.95 s	1.95s
search?keyword=00100	200	document	Other	11.6 kB	162 ms	162ms

Elastic Search를 사용하지 않은 이유

검색 병목 해결 이후, 추가 검색 엔진 도입 여부를 프로젝트 조건으로 판단

선택 흐름



도입 판단

01 기능 적합성

고급 검색 랭킹·분산 검색은 프로젝트 핵심 요구가 아님

02 운영 복잡도

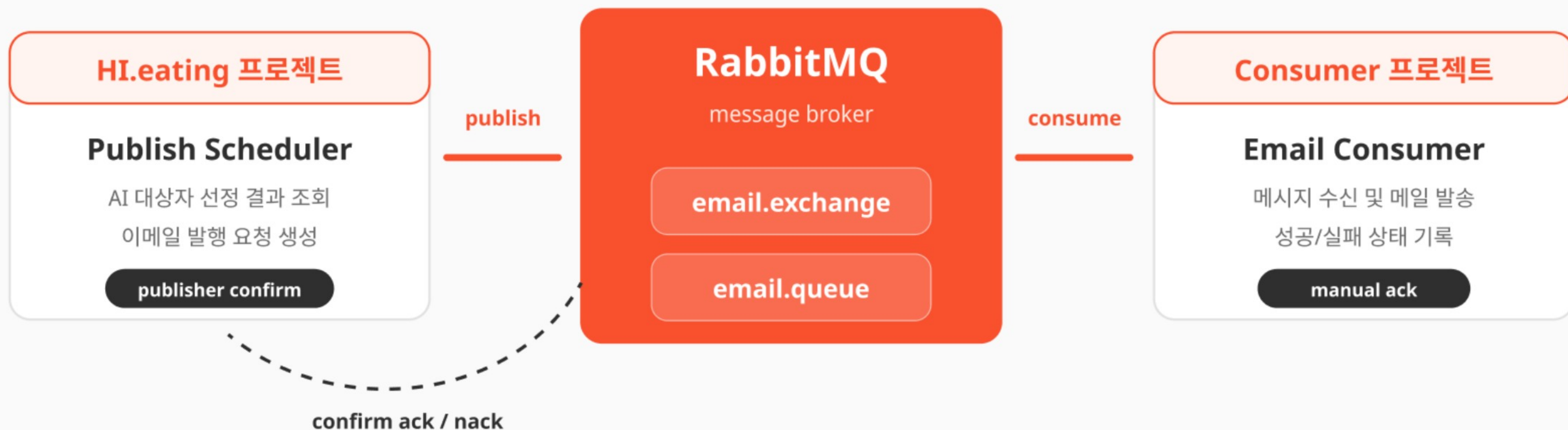
검색 클러스터·동기화·장애 대응 포인트 추가 MVP 범위 대비 관리 비용이 커짐

03 프로젝트 우선순위

상품/구매 자체보다 AI·비동기 파이프라인 검증이 중심
트렌드보다 상황에 맞는 기술 선택에 집중

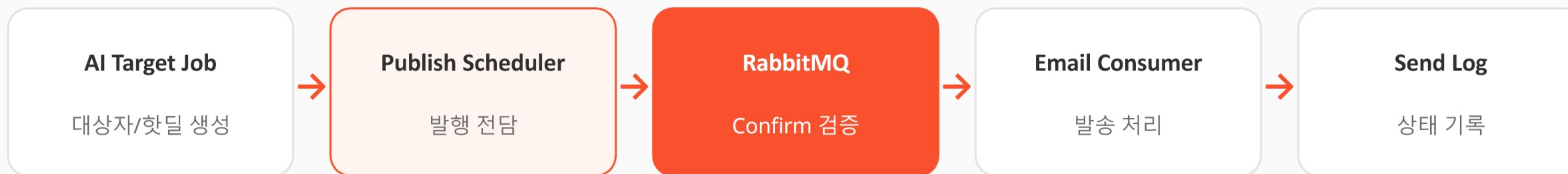
RabbitMQ 이메일 파이프라인 구성

Publisher - RabbitMQ - Consumer 분리



RabbitMQ 이메일 파이프라인 누락 검증/성능 개선

AI 로직에 묶였던 publish를 스케줄러로 분리해 장애 추적성과 처리 안정성을 개선



누락 검증

RabbitMQ Publisher Confirm으로 broker 수신 여부를 확인하고, 발송 로그 상태값으로 publish/consume 흐름을 대조

문제

AI 처리 흐름 안에서 publish가 함께 실행
장애 발생 시 AI 실패인지 MQ 발행 실패인지 원인 분리 어려움

개선

publish 전담 스케줄러 분리로 재시도·모니터링·장애 원인 추적이 쉬워지고 AI 처리와 발행 처리의 병목을 분리

RabbitMQ 이메일 파이프라인 구성

실제 이메일 발송 결과

N 메일

전체메일 5673 / 6589 안읽음 삭제

메일 검색 상세

답장 전체답장 전달 삭제 스팸신고 안읽음 이동 더보기

☆ (광고) [하이어팅] 관심 상품 [[체크오] 아르타민 부스트 패션후르츠 10*14포] 핫딜 알림

보낸사람 hieating1@gmail.com

받는사람 fig989@naver.com

2026년 7월 9일 (목) 오전 10:36

오늘의 핫딜

부스트 핫딜



안녕하세요, 유상록님 😊

놓치기 아쉬운 특별 핫딜이 도착했어요!

🔥 [체크오] 아르타민 부스트 패션후르츠 10*14포 🔥

지금 30% 할인된 20,790원에 만나보실 수 있습니다.

제목: '부스트 핫딜'에서 추천하는 '체크오 아르타민 부스트 패션후르츠 10*14포'는 건강식품으로 인기가 많고, 현재 30% 할인된 가격에 판매 중입니다. 이 제품은 건강을 유지하고 에너지를 충전할 수 있는 좋은 선택이 될 것입니다. 빠르게 구매해 보세요!

🕒 혜택 기간은 2026년 7월 12일까지입니다.

👉 인기 상품은 조기 소진될 수 있으니 서둘러 확인해보세요.

감사합니다.)

Hi.eating 드림

© NAVER Corp.

오늘의 핫딜

부스트 핫딜



안녕하세요, 유상록님 😊

놓치기 아쉬운 특별 핫딜이 도착했어요!

🔥 [체크오] 아르타민 부스트 패션후르츠 10*14포 🔥

지금 30% 할인된 20,790원에 만나보실 수 있습니다.

제목: '부스트 핫딜'에서 추천하는 '체크오 아르타민 부스트 패션후르츠 10*14포'는 건강식품으로 인기가 많고, 현재 30% 할인된 가격에 판매 중입니다. 이 제품은 건강을 유지하고 에너지를 충전할 수 있는 좋은 선택이 될 것입니다. 빠르게 구매해 보세요!

🕒 혜택 기간은 2026년 7월 12일까지입니다.

👉 인기 상품은 조기 소진될 수 있으니 서둘러 확인해보세요.

감사합니다.)

Hi.eating 드림

[체크오] 아르타민 부스트 패션후르츠 10*14포

29,700원

30% 할인 20,790원

핫딜 바로 보기

Kafka가 아닌 RabbitMQ를 사용한 이유

이메일 발송 파이프라인에는 대규모 스트리밍보다 명확한 작업 큐와 라우팅이 더 적합

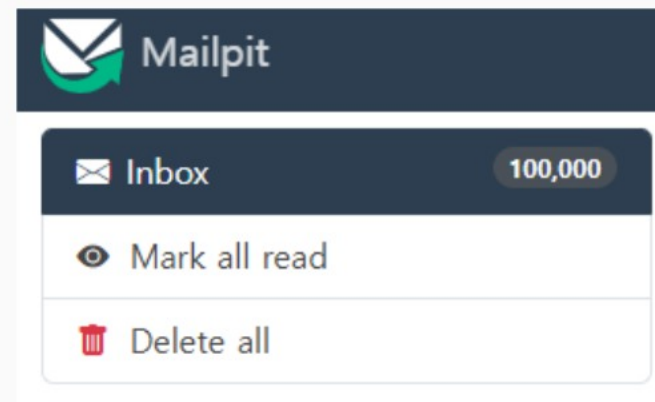
판단 기준: 메시지 규모 · 운영 복잡도 · 장애 대응 · 구현 목적

항목	RabbitMQ	Kafka
목적	작업 큐/이메일 발송에 적합	대규모 이벤트 스트리밍에 강점
운영	가볍고 구조가 단순	브로커/토픽 운영 부담 큼
라우팅	Exchange/Queue로 발송 흐름 명확	컨슈머 그룹/파티션 설계 필요
장애 대응	ACK/Confirm 기반 누락 검증 용이	처리량은 강하지만 설계 복잡도 증가

서비스의 이메일 발송 규모와 목적에는 Kafka보다 RabbitMQ의 단순한 작업 큐 모델이 더 현실적이었습니다.

RabbitMQ 이메일 발송 성능 측정 결과

2026-07-09 13:59:29 KST · Consumer 1대 기준



TPS

19.79 건/s

성공 발송

100,000 건

평균 발송 시간

45.57 ms

전체 완료 시간

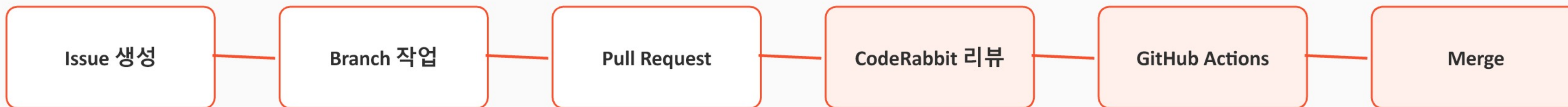
92분 38초

측정 항목	측정값	의미
TPS	19.79 건/s	초당 처리 건수
성공 건수	100,000건	정상 발송 처리
실패 건수	0건	SMTP 오류, 네트워크 오류 없음
평균 발송 시간	45.57ms	1건 처리 평균 시간
재시도 횟수	0회	실패 후 재처리 대상 없음

[Email LoadTest Consumer] progress processed=110000 sent=110000 failed=0 retrying=0 elapsedMs=5558156 tps=19.79 avgSendMs=45.57

협업 및 프로젝트 관리

PR 중심 개발 프로세스와 자동화 도구로 코드 품질과 커뮤니케이션 비용을 관리



Git Branch 전략

기능 단위 브랜치와 PR 기반 리뷰
흐름으로 변경 이력 추적

GitHub Actions

빌드/테스트 자동화로 리뷰 전
기본 품질 검증

CodeRabbit

AI PR 리뷰를 활용해 리뷰 누락 방지 및
소통 비용 절감

기술적 발견 및 결론

HI.eating은 기능 구현을 넘어, 제한된 프로젝트 환경 안에서
AI·검색·비동기 처리의 실무형 운영 구조를 검증한 프로젝트입니다.

1

로컬 AI의 가능성

경량 로컬 LLM도 프롬프트 설계와
백엔드 보정 정책을 결합하면 이메일
초안·리뷰 요약에 활용 가능

2

트렌드보다 문제에 맞는 기술 선택

Elasticsearch, Kafka, 외부 AI처럼
큰 기술을 무조건 도입하기보다,
현재 요구사항에 맞는 Oracle Text와
RabbitMQ, 로컬 AI가
더 현실적이라고 판단

3

백엔드 방어 중요성

AI 출력은 참고값으로 취급하고
검증·보정·재시도·로그 기반 복구를
백엔드에 설계

팀원 소개



유상록

상품 상세, 구매 기능, 마이페이지, 리뷰 시스템, 내 식탁 꾸미기, 상품 추천 시스템, 이미지 업로드 서버 구현



공희진

관리자 상품 관리, 핫딜 지정, Spring AI 기반 이메일 파이프라인



이재우

DB 스키마, Spring Security, WebSocket(채팅), MCP, 크롤러, 인프라



전형준

상품 조회, 검색 성능 개선, 즐겨찾기, 메인 페이지 구성, rabbitMQ 이메일 파이프라인

감사합니다.

질문 있으시면 말씀해 주세요.

HI.eating